Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №4
на тему

АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ. АЛГОРИТМ МАК-ЭЛИСА

БГУИР КП 6-05-0612-02 015 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф.Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 7
Приложение А (обязательное) Листинг программного кода 8

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается асимметричная криптосистема Мак-Элиса, предложенная Робертом Мак-Элисом в 1978 году. В отличие от криптосистем RSA и Рабина, стойкость которых основана на теоретико-числовых задачах, криптосистема Мак-Элиса опирается на теорию алгебраического кодирования. Её безопасность обеспечивается сложностью декодирования произвольного линейного кода, поэтому она относится к постквантовым криптосистемам.

Закрытый ключ содержит двоичный код Гоппы, исправляющий t ошибок, для которого известен эффективный алгоритм декодирования, а также случайную невырожденную матрицу S и перестановочную матрицу P. Открытым ключом является матрица, маскирующая код Гоппы под произвольный линейный код:

$G' = SGP$.

Зашифрование сообщения m длины k бит выполняется умножением на открытый ключ и добавлением случайного вектора ошибок z длины n, содержащего ровно t единиц:

$c = mG' + z$.

При расшифровании владелец закрытого ключа отменяет перестановку, исправляет ошибки с помощью алгоритма декодирования кода Гоппы и умножает результат на матрицу S^{-1}. Противник, которому известна только матрица G', вынужден решать задачу декодирования произвольного линейного кода, для которой эффективный алгоритм неизвестен.

Целью лабораторной работы является разработка программного средства на языке C++, реализующего генерацию ключей, зашифрование и расшифрование текстовых файлов с использованием криптосистемы Мак-Элиса для криптостойких размеров порождающей матрицы.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для работы с криптосистемой Мак-Элиса. Все алгоритмы реализованы без использования сторонних библиотек. Программа поддерживает генерацию ключевой пары, зашифрование файлов открытым ключом и расшифрование закрытым ключом.

В реализации используются параметры, рекомендованные для данной системы: длина кода $n = 1024$, число исправляемых ошибок $t = 38$, вычисления выполняются в поле $\mathrm{GF}(2^{10})$. Размерность кода составляет $k = n - mt = 1024 - 10 \cdot 38 = 644$, поэтому порождающая матрица имеет размер 644×1024.

Программа разделена на несколько модулей. Модуль gf2m реализует арифметику конечного поля $\mathrm{GF}(2^m)$. Элемент поля хранится как 10-битное число, сложение выполняется операцией исключающего ИЛИ, а умножение и деление – через таблицы логарифмов по примитивному элементу. Поле строится по примитивному многочлену $x^{10} + x^3 + 1$.

Модуль poly реализует многочлены с коэффициентами из поля $\mathrm{GF}(2^m)$: умножение, деление с остатком, расширенный алгоритм Евклида, нахождение обратного элемента и квадратного корня по модулю многочлена. Для генерации многочлена Гоппы используется тест неприводимости Бен-Ора: многочлен $g(x)$ степени t неприводим, если

$$\gcd\left(g(x), x^{q^i} - x\right) = 1, \quad i = 1, \ldots, \frac{t}{2}, \quad q = 2^m.$$

Модуль binmatrix содержит двоичные векторы и матрицы, упакованные по 64 бита в машинное слово. В нём реализованы умножение, приведение к ступенчатому виду методом Гаусса, обращение матрицы и нахождение базиса ядра.

Модуль gopp a строит двоичный код Гоппы по неприводимому многочлену $g(x)$ степени t и носителю $L = (\alpha_0, \ldots, \alpha_{n-1})$, в качестве которого используются все элементы поля в случайном порядке. Проверочная матрица формируется по правилу

$$H_{ji} = \frac{\alpha_i^j}{g(\alpha_i)}, \quad j = 0, \ldots, t-1,$$

после чего каждый элемент поля раскладывается в столбец из 10 бит, и получается двоичная матрица размером 380×1024. Порождающая матрица G вычисляется как базис ядра матрицы H.

Генерация ключей выполняется функцией generateKeys. Формируется случайный код Гоппы, случайная невырожденная матрица S размером 644×644 вместе с обратной матрицей и случайная перестановка P. Открытый ключ содержит матрицу $G' = SGP$, закрытый ключ – многочлен $g(x)$, носитель L, перестановку P и матрицу S^{-1}. Для получения случайных значений используется криптостойкий генератор операционной системы. Размер открытого ключа составляет 82 452 байта, закрытого – 58 410 байт.

Перед зашифрованием исходный файл разбивается на блоки по 80 байт, которые дополняются случайными битами до длины $k = 644$ бита. Каждый блок зашифровывается по формуле $c = mG' + z$, где вектор ошибок z содержит

ровно 38 единиц в случайных позициях. Блок шифртекста имеет длину 1024 бита, поэтому размер данных при шифровании увеличивается примерно в 1,6 раза. В начало зашифрованного файла записывается сигнатура MCE1, параметры кода и исходный размер файла.

Расшифрование блока начинается с отмены перестановки: $cP^{-1} = (mS)G + zP^{-1}$. Вес вектора ошибок при перестановке не изменяется, поэтому полученный вектор отличается от кодового слова не более чем в 38 позициях. Для исправления ошибок используется алгоритм Паттерсона. Сначала вычисляется синдромный многочлен

$$S(x) = \sum_{i:r_i=1} \frac{1}{x-\alpha_i} \bmod g(x).$$

Затем находятся $T(x) = S(x)^{-1} \bmod g(x)$ и $\tau(x) = \sqrt{T(x) + x} \bmod g(x)$. Расширенный алгоритм Евклида для пары (g, τ) даёт многочлены $a(x)$ и $b(x)$ малой степени, из которых строится многочлен локаторов ошибок:

$$\sigma(x) = a^2(x) + xb^2(x).$$

Корни многочлена $\sigma(x)$ среди элементов носителя указывают позиции ошибок. После их исправления из информационных позиций кодового слова извлекается вектор mS, и исходный блок восстанавливается умножением $m = (mS)S^{-1}$. Если число найденных корней не совпадает со степенью $\sigma(x)$, блок считается повреждённым, и программа сообщает об ошибке.

Для запуска программы предусмотрены четыре операции:

```
mceliece keygen public.key private.key
mceliece encrypt public.key input.txt secret.enc
mceliece decrypt private.key secret.enc output.txt
mceliece compare input.txt output.txt
```

Первая команда создаёт ключевую пару, вторая выполняет зашифрование файла, третья – его расшифрование, а четвёртая сравнивает исходный и восстановленный файлы. При генерации ключей программа выводит многочлен Гоппы, фрагменты носителя и перестановки, а также фрагменты матриц G, S^{-1} и G', что продемонстрировано на рисунке 1.

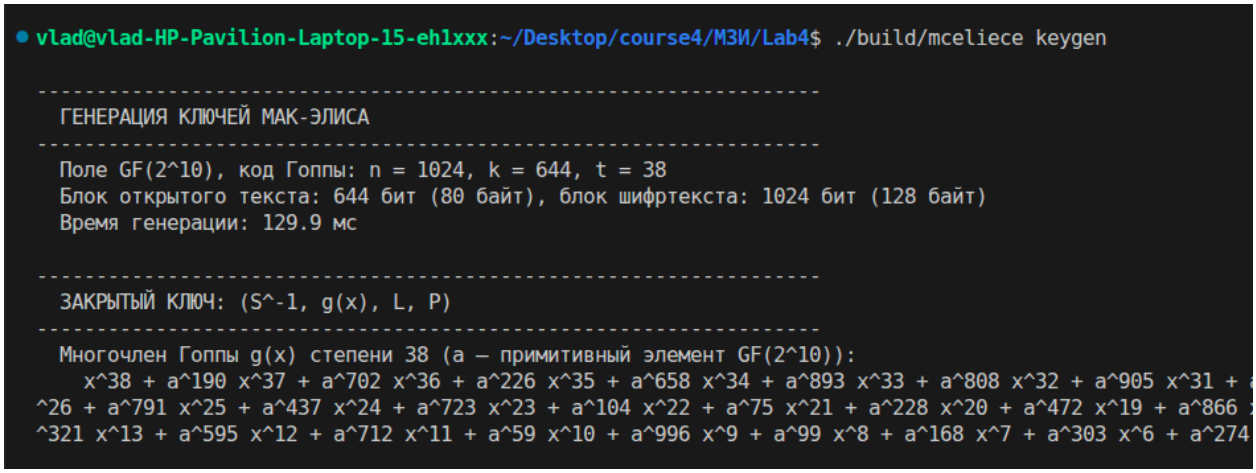


Рисунок 1.1 – Вывод программы при генерации ключевой пары

При зашифровании и расшифровании выводится подробный разбор первого блока: исходный вектор m, произведение mG', вектор ошибок z и шифртекст c, а при расшифровании – вектор cP^{-1}, степени синдрома и многочлена локаторов, позиции исправленных ошибок и восстановленный вектор m.

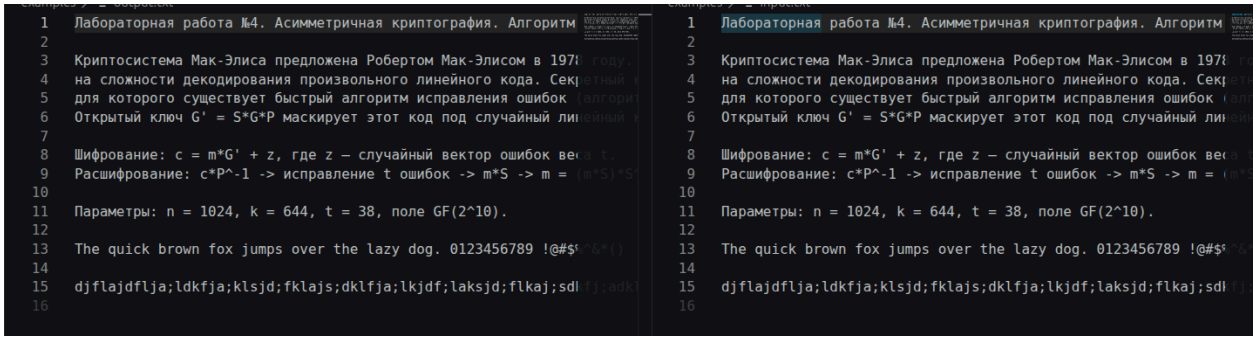


Рисунок 1.2 – Тексты в исходном и расшифрованном файлах

На рисунке 1.2 продемонстрирован корректный результат работы программы: текст в файле output.txt совпадает с текстом в файле input.txt.

Таким образом, в ходе работы была реализована криптосистема Мак-Элиса на основе двоичного кода Гоппы с параметрами $(1024, 644, 38)$ и алгоритма декодирования Паттерсона с поддержкой работы с файлами.

ВЫВОД

В ходе выполнения лабораторной работы была изучена криптосистема Мак-Элиса и разработано программное средство на языке C++, реализующее генерацию ключевой пары, зашифрование и расшифрование файлов.

В программе реализованы арифметика конечного поля $\mathrm{GF}(2^{10})$, операции над многочленами и двоичными матрицами, построение двоичного кода Гоппы по неприводимому многочлену степени 38 и декодирование алгоритмом Паттерсона. Код Гоппы маскируется под произвольный линейный код с помощью невырожденной матрицы S и перестановки P.

Проведённое тестирование показало, что после последовательного зашифрования и расшифрования исходное содержимое файла восстанавливается полностью, а при повреждении шифртекста или использовании чужого ключа расшифрование завершается сообщением об ошибке. Подтверждены и известные недостатки системы: большой размер открытого ключа, около 80 КБ, и увеличение размера данных при шифровании примерно в 1,6 раза.

В результате выполнения лабораторной работы были получены практические навыки реализации асимметричного криптографического алгоритма Мак-Элиса, работы с конечными полями, многочленами и двоичными матрицами, а также использования кодов, исправляющих ошибки, в криптографии.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – mceliece.h

```
#pragma once

#include "binmatrix.h"
#include "gf2m.h"
#include "goppa.h"
#include "random.h"

#include <cstdint>
#include <memory>
#include <string>
#include <vector>

namespace mce {

struct Params {
    int    m = 10;
    int    t = 38;
    size_t n = 1024;
};


struct EncryptTrace {
    BitVec msg;
    BitVec error;
    BitVec cipher;
};

struct DecryptTrace {
    BitVec cipher;
    BitVec permuted;
    int    syndromeDeg = -1;
    int    sigmaDeg = -1;
    BitVec error;
    BitVec mS;
    BitVec msg;
};


struct PublicKey {
    int    m = 0, t = 0;
    size_t n = 0, k = 0;
    BitMatrix Gpub;

    size_t plainBlockBytes() const { return k / 8; }
    size_t cipherBlockBytes() const { return (n + 7) / 8; }

    BitVec encryptBlock(const BitVec& msg, Rng& rng, EncryptTrace* trace =
nullptr) const;

    void save(const std::string& path) const;
    static PublicKey load(const std::string& path);
};

struct PrivateKey {
    int      m = 0, t = 0;
    size_t   n = 0, k = 0;
```

```
    uint32_t fieldPoly = 0;
    std::vector<uint16_t> g;
    std::vector<uint16_t> support;
    std::vector<uint32_t> perm;
    BitMatrix Sinv;

    void save(const std::string& path) const;
    static PrivateKey load(const std::string& path);
};

void generateKeys(const Params& p, Rng& rng, PublicKey& pub, PrivateKey&
priv);

class Decryptor {
public:
    explicit Decryptor(const PrivateKey& key);

    BitVec decryptBlock(const BitVec& c, DecryptTrace* trace = nullptr)
const;

    const PrivateKey& key()  const { return key_; }
    const GoppaCode&  code() const { return *code_; }

private:
    PrivateKey key_;
    std::unique_ptr<GF2m> field_;
    std::unique_ptr<GoppaCode> code_;
};

std::vector<uint8_t> encryptBytes(const PublicKey& pub, const
std::vector<uint8_t>& data, Rng& rng,
                                  EncryptTrace* firstBlock = nullptr);
std::vector<uint8_t> decryptBytes(const Decryptor& dec, const
std::vector<uint8_t>& cipher,
                                  DecryptTrace* firstBlock = nullptr);

std::vector<uint8_t> readFile(const std::string& path);
void writeFile(const std::string& path, const std::vector<uint8_t>& data);

}  // namespace mce
```

,

Листинг А.2 – mceliece.cpp

```
#include "mceliece.h"

#include <cstring>
#include <fstream>
#include <iterator>
#include <stdexcept>

namespace mce {

namespace {

class Writer {
public:
    void bytes(const void* p, size_t n) {
        auto* b = static_cast<const uint8_t*>(p);
        buf.insert(buf.end(), b, b + n);
    }
```

```
    void u16(uint16_t v) { for (int i = 0; i < 2; ++i)
buf.push_back(uint8_t(v >> (8 * i))); }
    void u32(uint32_t v) { for (int i = 0; i < 4; ++i)
buf.push_back(uint8_t(v >> (8 * i))); }
    void u64(uint64_t v) { for (int i = 0; i < 8; ++i)
buf.push_back(uint8_t(v >> (8 * i))); }
    void bitvec(const BitVec& v) { auto b = v.toBytes(); bytes(b.data(),
b.size()); }
    std::vector<uint8_t> buf;
};

class Reader {
public:
    Reader(const std::vector<uint8_t>& d, const char* what) : d_(d),
what_(what) {}
    const uint8_t* take(size_t n) {
        if (n > d_.size() - pos_) fail("файл обрезан");
        const uint8_t* p = d_.data() + pos_;
        pos_ += n;
        return p;
    }
    uint64_t le(int n) { const uint8_t* p = take(n); uint64_t v = 0; for (int
i = 0; i < n; ++i) v |= uint64_t(p[i]) << (8 * i); return v; }
    uint16_t u16() { return uint16_t(le(2)); }
    uint32_t u32() { return uint32_t(le(4)); }
    uint64_t u64() { return le(8); }
    BitVec bitvec(size_t nbits) { return BitVec::fromBytes(take((nbits + 7) /
8), nbits); }
    void magic(const char* m) { if (std::memcmp(take(4), m, 4) != 0)
fail("неверная сигнатура"); }
    void end() const { if (pos_ != d_.size()) fail("лишние данные в
конце"); }
    size_t remaining() const { return d_.size() - pos_; }
    [[noreturn]] void fail(const std::string& msg) const {
        throw std::runtime_error(std::string(what_) + ": " + msg);
    }
private:
    const std::vector<uint8_t>& d_;
    size_t pos_ = 0;
    const char* what_;
};

void checkParams(int m, int t, size_t n, size_t k, const Reader& r) {
    if (m < 2 || m > 16) r.fail("недопустимое m");
    if (n < 2 || n > (size_t(1) << m)) r.fail("недопустимое n");
    if (t < 1 || size_t(m) * t >= n) r.fail("недопустимое t");
    if (k == 0 || k > n) r.fail("недопустимое k");
}

}  // namespace
std::vector<uint8_t> readFile(const std::string& path) {
    std::ifstream in(path, std::ios::binary);
    if (!in) throw std::runtime_error("не удаётся открыть файл: " + path);
    return {std::istreambuf_iterator<char>(in),
std::istreambuf_iterator<char>()};
}
void writeFile(const std::string& path, const std::vector<uint8_t>& data) {
    std::ofstream out(path, std::ios::binary);
    if (!out) throw std::runtime_error("не удаётся создать файл: " + path);
    out.write(reinterpret_cast<const char*>(data.data()),
static_cast<std::streamsize>(data.size()));
    if (!out) throw std::runtime_error("ошибка записи в файл: " + path);
```

```
}

void generateKeys(const Params& p, Rng& rng, PublicKey& pub, PrivateKey&
priv) {
    GF2m field(p.m);
    if (p.n > field.size()) throw std::invalid_argument("generateKeys: n >
2^m");
    if (size_t(p.m) * p.t >= p.n) throw std::invalid_argument("generateKeys:
m·t >= n, код пустой");

    GoppaCode code = GoppaCode::generate(field, p.t, p.n, rng);
    const size_t k = code.k();

    BitMatrix Sinv;
    BitMatrix S = BitMatrix::randomInvertible(k, rng, Sinv);

    std::vector<uint32_t> perm = randomPermutation(p.n, rng);

    pub.m = p.m; pub.t = p.t; pub.n = p.n; pub.k = k;
    pub.Gpub = (S * code.generator()).permuteColumns(perm);

    priv.m = p.m; priv.t = p.t; priv.n = p.n; priv.k = k;
    priv.fieldPoly = field.poly();
    priv.g.assign(code.goppaPoly().coeffs().begin(),
code.goppaPoly().coeffs().end());
    priv.support.assign(code.support().begin(), code.support().end());
    priv.perm = std::move(perm);
    priv.Sinv = std::move(Sinv);
}

BitVec PublicKey::encryptBlock(const BitVec& msg, Rng& rng, EncryptTrace*
trace) const {
    if (msg.size() != k) throw std::invalid_argument("encryptBlock: длина
блока должна быть k");
    const BitVec z = BitVec::randomOfWeight(n, static_cast<size_t>(t), rng);
    BitVec c = Gpub.mulLeft(msg);                       // m · G'
    c ^= z;                                             // + z
    if (trace) *trace = {msg, z, c};
    return c;
}

Decryptor::Decryptor(const PrivateKey& key) : key_(key) {
    field_ = std::make_unique<GF2m>(key.m, key.fieldPoly);
    std::vector<GF2m::Elem> gc(key.g.begin(), key.g.end());
    std::vector<GF2m::Elem> L(key.support.begin(), key.support.end());
    code_ = std::make_unique<GoppaCode>(*field_, Poly(*field_,
std::move(gc)), std::move(L));
    if (code_->k() != key.k || code_->t() != key.t)
        throw std::runtime_error("закрытый ключ повреждён: параметры кода не
совпадают");
}

BitVec Decryptor::decryptBlock(const BitVec& c, DecryptTrace* trace) const {
    if (c.size() != key_.n) throw std::invalid_argument("decryptBlock: длина
блока должна быть n");
    BitVec c1 = c.unpermuted(key_.perm);
    if (trace) {
        const Poly S = code_->syndrome(c1);
        trace->cipher = c;
        trace->permuted = c1;
        trace->syndromeDeg = S.degree();
        trace->sigmaDeg = code_->errorLocator(S).degree();
```

```
    }
    BitVec codeword, error;
    if (!code_->decode(c1, codeword, error))
        throw std::runtime_error("блок не декодируется: шифртекст повреждён
или ключ не тот");
    BitVec mS = code_->extractMessage(codeword);
    BitVec m = key_.Sinv.mulLeft(mS);
    if (trace) {
        trace->error = error;
        trace->mS = mS;
        trace->msg = m;
    }
    return m;
}

void PublicKey::save(const std::string& path) const {
    Writer w;
    w.bytes("MCPK", 4);
    w.u32(m); w.u32(t); w.u32(uint32_t(n)); w.u32(uint32_t(k));
    for (size_t i = 0; i < k; ++i) w.bitvec(Gpub.row(i));
    writeFile(path, w.buf);
}

PublicKey PublicKey::load(const std::string& path) {
    auto data = readFile(path);
    Reader r(data, "открытый ключ");
    r.magic("MCPK");
    PublicKey p;
    p.m = int(r.u32()); p.t = int(r.u32()); p.n = r.u32(); p.k = r.u32();
    checkParams(p.m, p.t, p.n, p.k, r);
    p.Gpub = BitMatrix(p.k, p.n);
    for (size_t i = 0; i < p.k; ++i) p.Gpub.row(i) = r.bitvec(p.n);
    r.end();
    return p;
}

void PrivateKey::save(const std::string& path) const {
    Writer w;
    w.bytes("MCSK", 4);
    w.u32(m); w.u32(t); w.u32(uint32_t(n)); w.u32(uint32_t(k));
w.u32(fieldPoly);
    for (auto c : g) w.u16(c);
    for (auto a : support) w.u16(a);
    for (auto p : perm) w.u32(p);
    for (size_t i = 0; i < k; ++i) w.bitvec(Sinv.row(i));
    writeFile(path, w.buf);
}

PrivateKey PrivateKey::load(const std::string& path) {
    auto data = readFile(path);
    Reader r(data, "закрытый ключ");
    r.magic("MCSK");
    PrivateKey p;
    p.m = int(r.u32()); p.t = int(r.u32()); p.n = r.u32(); p.k = r.u32();
    p.fieldPoly = r.u32();
    checkParams(p.m, p.t, p.n, p.k, r);
    p.g.resize(p.t + 1);
    for (auto& c : p.g) c = r.u16();
    p.support.resize(p.n);
    for (auto& a : p.support) a = r.u16();
    p.perm.resize(p.n);
    std::vector<bool> seen(p.n, false);
```

```
    for (auto& x : p.perm) {
        x = r.u32();
        if (x >= p.n || seen[x]) r.fail("P не является перестановкой");
        seen[x] = true;
    }
    p.Sinv = BitMatrix(p.k, p.k);
    for (size_t i = 0; i < p.k; ++i) p.Sinv.row(i) = r.bitvec(p.k);
    r.end();
    return p;
}
std::vector<uint8_t> encryptBytes(const PublicKey& pub, const
std::vector<uint8_t>& data, Rng& rng,
                                  EncryptTrace* firstBlock) {
    const size_t pb = pub.plainBlockBytes();
    if (pb == 0) throw std::invalid_argument("encryptBytes: k < 8");
    const size_t blocks = (data.size() + pb - 1) / pb;

    Writer w;
    w.bytes("MCE1", 4);
    w.u32(uint32_t(pub.n)); w.u32(uint32_t(pub.k)); w.u64(data.size());
    w.buf.reserve(w.buf.size() + blocks * pub.cipherBlockBytes());

    for (size_t b = 0; b < blocks; ++b) {
        const size_t off = b * pb;
        const size_t len = std::min(pb, data.size() - off);
        BitVec msg = BitVec::random(pub.k, rng);
        for (size_t i = 0; i < len; ++i)
            for (int bit = 0; bit < 8; ++bit)
                msg.set(8 * i + bit, data[off + i] >> bit & 1);
        w.bitvec(pub.encryptBlock(msg, rng, b == 0 ? firstBlock : nullptr));
    }
    return w.buf;
}
std::vector<uint8_t> decryptBytes(const Decryptor& dec, const
std::vector<uint8_t>& cipher,
                                  DecryptTrace* firstBlock) {
    Reader r(cipher, "шифртекст");
    r.magic("MCE1");
    const size_t n = r.u32(), k = r.u32();
    const uint64_t len = r.u64();
    if (n != dec.key().n || k != dec.key().k)
        r.fail("параметры шифртекста не совпадают с ключом");
    const size_t pb = k / 8, cb = (n + 7) / 8;
    const uint64_t blocks = len / pb + (len % pb != 0);   // без переполнения
при огромном len
    if (r.remaining() % cb != 0 || r.remaining() / cb != blocks)
        r.fail("неверное число блоков");

    std::vector<uint8_t> out;
    out.reserve(len);
    for (size_t b = 0; b < blocks; ++b) {
        BitVec msg = dec.decryptBlock(r.bitvec(n), b == 0 ? firstBlock :
nullptr);
        const size_t take = std::min<uint64_t>(pb, len - out.size());
        auto bytes = msg.toBytes();
        out.insert(out.end(), bytes.begin(), bytes.begin() + take);
    }
    return out;
}

}  // namespace mce
```